

Лабораторна робота №4. Нейронні мережі та згорткові архітектури на CIFAR-10

25 серпня 2026 р.

Мета

Реалізувати багатошаровий перцептрон двічі — власноруч на NumPy зі своїм зворотним поширенням і той самий у PyTorch — та довести їхню чисельну еквівалентність; далі побудувати згорткову мережу на CIFAR-10 і виміряти, скільки насправді дає кожен інгредієнт сучасного рецепта: архітектура, ініціалізація, нормалізація та аугментація.

Теоретичні відомості

CIFAR-10 — 60 000 кольорових фотографій 32×32 у десяти класах. На відміну від рукописних цифр, тут об'єкт може бути під будь-яким кутом, на будь-якому тлі й будь-якого кольору, тому повнозв'язна мережа впирається приблизно в 50 % правильних відповідей, а різниця між нею та згортковою стає очевидною, а не косметичною.

- **Зворотне поширення.** Градієнт за всіма параметрами рахується одним проходом назад за ланцюговим правилом, тому коштує приблизно стільки ж, скільки прямий прохід. Реалізацію завжди перевіряють скінченними різницями: помилка в градієнті виглядає як «модель погано навчається» і легко приймається за невдалий темп навчання.
- **Активації та зникаючий градієнт.** Похідна сигмоїди не перевищує $1/4$, тож у мережі з десяти шарів множник сягає 10^{-6} і перші шари майже не отримують сигналу. ReLU має похідну 1 на додатній півосі й цієї проблеми не створює, натомість може «вимикати» нейрони назавжди.
- **Згортковий шар.** Замість зв'язку кожного входу з кожним виходом застосовується невелике ядро в кожній позиції. Кількість ваг $C_{out}(C_{in}K^2 +$

1) не залежить від розміру зображення, а спільні ваги дають еквіваріантність до зсуву. Для входу $32 \times 32 \times 3$ економія проти повнозв'язного шару — два порядки.

- **Регуляризація глибоких мереж.** Пакетна нормалізація стабілізує масштаби активацій усередині мережі, dropout заважає нейронам покладатися на конкретних сусідів, аугментація породжує нові приклади перетвореннями, що зберігають мітку. На CIFAR-10 аугментація дає найбільший приріст із трьох, і це варто виміряти, а не прийняти на віру.

Корисні посилання для ознайомлення:

- [CIFAR-10](#) — опис набору; вантажитися через `torchvision.datasets.CIFAR10`;
- [Stanford CS231n](#) — конспекти лекцій із поясненням кожного шару; курс побудований саме навколо CIFAR-10;
- [Dive into Deep Learning](#) — підручник із виконуваним кодом на PyTorch;
- [PyTorch: Training a Classifier](#) — офіційний приклад саме на цьому наборі;
- [Google Colab](#) — безкоштовний GPU; на процесорі навчання займе десятки хвилин замість одиниць.

Порядок виконання

1. Обрати фреймворк: `pytorch` або `tensorflow` (далі наведено назви для PyTorch). Створити оточення, встановити `torch`, `torchvision`, `numpy`, `matplotlib`, `scikit-learn` і перевірити, який пристрій доступний. Якщо GPU немає, працювати в Colab.
2. Завантажити CIFAR-10 через `torchvision.datasets.CIFAR10` і підтвердити розміри: 50 000 тренувальних і 10 000 тестових зображень $32 \times 32 \times 3$, десять класів. Архів важить 170 МБ: на повільному з'єднанні завантаження триває довго, а перерване залишає пошкоджений файл, який доводиться качати наново. У Colab набір з'являється за секунди. Показати сітку випадкових зразків із підписами класів і розподіл класів. Навести базову лінію — логістичну регресію на сирих пікселях.
3. Реалізувати на NumPy мережу $3072 \rightarrow 256 \rightarrow 10$ з активацією ReLU, softmax і перехресною ентропією: прямий прохід, зворотне поширення й крок оновлення окремими функціями. Перевірити градієнт скінченими різницями і навести максимальну відносну розбіжність.
4. Зібрати ту саму архітектуру засобами `torch.nn`, скопіювати в неї початкові ваги реалізації з попереднього пункту й довести еквівалентність: після одного кроку оптимізації втрати обох реалізацій мають збігатися до похибки округлення.

5. Порівняти чотири конфігурації повнозв'язної мережі: дві активації (`sigmoid` і `relu`) на двох ініціалізаціях (Хе та Ксав'є) з однаковим бюджетом епох, темпом навчання й зерном. Побудувати криві навчання на одному графіку та пояснити, яка пара і чому навчається найшвидше.
6. Реалізувати згорткову мережу: `conv(32, 3x3) → ReLU → conv(32, 3x3) → ReLU → maxpool(2x2) → conv(64, 3x3) → ReLU → maxpool(2x2) → fc(256) → fc(10)`. Спершу порахувати кількість параметрів кожного шару вручну за формулою $C_{out}(C_{in}K^2 + 1)$ і форми тензора після кожного шару, потім звірити з `sum(p.numel() for p in model.parameters())`.
7. Порівняти повнозв'язну та згорткову мережі за трьома числами: кількість параметрів, час епохи, точність на тесті. Пояснити, чому згорткова виграє, попри меншу кількість ваг.
8. Порівняти чотири конфігурації згорткової мережі — без обох механізмів, лише з пакетною нормалізацією, лише з dropout, з обома. Навести таблицю з тестовою точністю й часом епохи.
9. Порівняти три режими аугментації: без неї, помірна (`RandomCrop(32, padding=4)` і `RandomHorizontalFlip`) і надмірна (додати `RandomVerticalFlip` та поворот до 90°). Пояснити, чому горизонтальне віддзеркалення тут доречне, а вертикальне — ні.
10. Для двох розмірів тренувальної вибірки — 5000 і 50 000 — побудувати криві тренувальної й валідаційної втрати. Визначити, у якому випадку потрібна рання зупинка, і показати епоху, на якій вона спрацювала б.
11. Побудувати матрицю похибок найкращої моделі. Знайти пару класів, які плутаються найчастіше, показати кілька таких зображень і пояснити, що саме їх зближує.
12. Візуалізувати вивчені ядра першого згорткового шару. Порівняти їх із вагами першого шару повнозв'язної мережі з пункту 3 і сказати, що змінилося.
13. Звести всі моделі в одну таблицю «модель — параметри — час епохи — тестова точність — приріст до базової лінії». Зробити висновки про те, який прийом дав найбільший внесок.

Додаткові завдання (необов'язкові)

Ці пункти виходять за межі аудиторного часу й виконуються за бажанням; вони не впливають на основну оцінку, але зараховуються як бонус.

1. Розширити пункт 5 до повної сітки: чотири активації (`sigmoid`, `tanh`, `relu`, `gelu`) на двох ініціалізаціях — вісім конфігурацій на одному графіку.

2. Додати до згорткової мережі залишкові з'єднання (один блок `ResidualBlock`) і порівняти збіжність із мережею без них за однакової глибини.
3. Узяти `resnet18` із `torchvision.models` з попередньо навченими вагами, донавчити останній шар на CIFAR-10 і порівняти з власною мережею за точністю та часом.

Контрольні запитання

1. Що станеться з багатошаровою мережею, якщо активацію замінити тотожною функцією?
2. Ви реалізували зворотне поширення, і модель не навчається. Як за одну перевірку відрізнити помилку в градієнті від невдалого темпу навчання?
3. Похідна сигмоїди не перевищує $1/4$. Яке число це дає для мережі з десяти шарів і що з цього впливає?
4. Скільки параметрів має згортковий шар із 64 фільтрами 3×3 над входом із 32 каналів, і чому це число не залежить від розміру зображення?
5. Чому на CIFAR-10 різниця між повнозв'язною та згортковою мережею набагато більша, ніж на рукописних цифрах?
6. Горизонтальне віддзеркалення для CIFAR-10 доречне, а вертикальне — ні. Чому, і на якому наборі було б навпаки?
7. Мережа дала 75 % на CIFAR-10. Чого ще бракує, щоб назвати результат хорошим?